Rank-One Self-Edits Transfer Within a Problem Family, Not Across Tasks: Finding the Decisive Token Is Not What Makes Them Work

Avijit Bhuin
Independent Researcher

Abstract

Can a language model repair its own failures by directly editing its weights, and do such repairs generalize? We study gradient-free, rank-one edits to a small code model (Qwen3.5-0.8B). In a pilot, model-chosen neuron edits fixed 5 of 45 failures, exactly as many as random edits. We then use an external diagnosis. Given a failing greedy answer and one of the model's own passing samples, a binary search over forced prefixes finds the *decisive token*, and a rank-one MLP edit flips it. On MBPP+, these edits fix 50 of 91 solvable failures, but edits at random positions where the model disagrees with the passing answer fix 47 (p=0.72), and summed edits do not transfer to held-out tasks. To tell "edits cannot transfer" apart from "these tasks share nothing to transfer", we build 32 parametric code templates whose variants differ only in the values in the specification. Here transfer appears. Across 16 templates and three splits, summing the fixing edits from five variants of a template raises greedy passes on its unseen variants from 44 to 217 of 292, breaking one. Norm-matched noise and edits from other templates have no reliable effect (paired $p < 0.001$ against both), and random-position edits transfer as well as decisive ones. A third of the fixing edits flip the first statement of the function body. Their key activations are nearly identical across variants of a template (median cosine 0.89) and dissimilar across templates (0.22). We conclude that rank-one self-edits act as context-keyed strategy switches: they generalize within a problem family but not across unrelated tasks, and precise localization is not what makes them work.

1 Introduction

Language models can increasingly inspect and modify neural networks, which raises an old question in a new form: can a model improve itself by editing its own weights? Knowledge editing shows that individual facts can be rewritten with small, targeted updates [Meng et al., 2022, 2023, Mitchell et al., 2022, Fang et al., 2025]. Self-adapting models show that a model can generate its own fine-tuning data and learn to do so with reinforcement learning [Zweiger et al., 2025]. Both lines suggest a tempting recipe for *skills* rather than facts: when the model fails a task, locate the faulty computation and edit it directly.

We test this recipe in its simplest form, on code generation, where success is checked automatically by hidden tests. Our question is not whether an edit can make a single task pass. With enough freedom almost any perturbation can. We ask two things instead. Does locating the right place to edit buy anything over editing somewhere else? And do the resulting edits generalize to problems they were not built on? We answer the second question at two distances: unrelated tasks (a random half of MBPP+) and unseen variants of the same problem (new values in the same specification).

Contributions.

- A deterministic *decisive-token* search: a binary search over forced prefixes of a passing sample that finds the single decision separating a failing greedy answer from a passing one. It requires no sampling and about $\log_2 L$ greedy rollouts.
- A controlled comparison of rank-one edits at decisive tokens against random positions, random directions, norm-matched noise, and edits built for other problems, evaluated on per-task repair and on transfer.
- A benchmark of 32 parametric code templates for near transfer, and a positive result on it. Edits transfer to unseen variants of the same problem, specifically (neither noise nor other problems' edits help), and without needing the decisive token.
- A mechanistic account that reconciles the two transfer results. Fixing edits often switch the solution strategy at the start of the function body, keyed on a context that variants share and unrelated tasks do not. On MBPP+, where no such shared context exists, summed edits interfere instead.

2 Setup

Model and tasks. We use Qwen3.5-0.8B [Qwen Team, 2026] in bf16 on Apple MLX, with thinking mode disabled and greedy decoding (at most 1024 new tokens). Tasks are the 378 problems of MBPP+ [Austin et al., 2021, Liu et al., 2023]. The prompt shows the problem and its three original asserts (*visible tests*) and asks for the function only, without comments, docstrings, or tests. An answer *passes* if it passes both the visible tests and the full MBPP+ hidden suite. We exclude task 590, whose reference solution fails its own hidden suite, leaving 377 tasks. Greedy decoding passes 130 of them.

Pairs. For each task we draw 10 samples at temperature 0.7. A *pair* is a task whose greedy answer fails while at least one sample passes; there are 91. Among pairs, the mean sample pass rate is 0.27 (median 0.2). For each pair we choose, as the reference passing answer s, the passing sample that shares the longest prefix with the greedy answer.

Edit primitive. Let $a \in \mathbb{R}^{d_{\text{ff}}}$ be the input to the MLP down-projection W of layer ℓ at the last position of a context, and let r and w be the desired and the currently preferred next tokens. With unembedding E and final-norm gain g, we set $u = \frac{(E_r - E_w) \odot g}{\|(E_r - E_w) \odot g\|}$ and apply

$$W \leftarrow W + \alpha \, \frac{u \, a^\top}{\|a\|^2}, \tag{1}$$

which adds exactly αu to the MLP output for input a. Starting from α=1, we double α until r beats w by at least one logit at that context. We use ℓ=16 throughout. This is a deliberately simple, ROME-like rank-one update [Meng et al., 2022]. It has no key covariance, so it sets a lower bound on how local such an edit can be.

Method (3 attempts)	Fixed / 45 failures
Resample (T=0.7)	8
Error feedback in context	3
Model-chosen neuron scaling	5
Random neuron scaling	5

Table 1: Pilot on the first 100 MBPP+ tasks. The denominator is greedy answers that fail the visible tests; a fix means the final answer passes the hidden suite.

3 Pilot: the model chooses its own edit

Our first design gave the model control. After a failed attempt on the visible tests, we recorded the most active MLP neurons in layers 16–23 on the failing reply and labelled each with the tokens it promotes and suppresses under the logit lens [nostalgebraist, 2020, Geva et al., 2021]. We then asked the model to propose up to five neuron rescalings as JSON. Each edit was applied and the answer regenerated. Up to three rounds were allowed, and the weights were restored after every task. The control applied random rescalings to the same candidate neurons on the same schedule. Table 1 shows the outcome on the first 100 tasks. Model-chosen edits match random edits exactly and lose to simple resampling. The model cannot read its own neuron labels well enough to pick useful edits. The rest of the paper therefore asks a more basic question: does *any* localization, even a perfect external one, help?

4 Locating the decisive token

Let y be the failing greedy answer and s the reference passing answer. For $k \in \{0, \ldots, |s|\}$, let $G(k)$ be the outcome of forcing the prefix $s_{<k}$ and continuing greedily. By construction $G(0)$ fails and $G(|s|)$ passes. Binary search finds an index k^* with $G(k^*)$ failing and $G(k^*{+}1)$ passing, using about $\log_2 |s|$ greedy rollouts. At k^* the model's greedy choice w differs from $s_{k^*} = r$, and taking r instead is enough to turn the outcome. This is a deterministic, single-trajectory relative of pivotal-token search [Abdin et al., 2024], which estimates success probabilities by sampling.

The simpler alternative, the first position where y and s diverge, almost always lands on cosmetic tokens such as comment wording or docstring phrasing. We found the same for sampling-based pivotal tokens, which motivated both the code-only prompt and the decisive search.

Episodes. An episode applies up to three edits. At each step we find the decisive token under the *current* weights, flip it with the primitive above, keep the edit, and test whether greedy decoding from the bare prompt now passes. The episode stops early if the decisive search finds no bracket, or if r and w decode to the same text (e.g. two tokenizations of `int`). All edits are rolled back after the episode.

Controls. *Random position*: one forward pass on the prompt plus s lists every position where the model's argmax disagrees with s. We pick one uniformly at random and flip it in the same way. This control keeps the target answer, the direction family, and the stopping rule, and removes only the localization. *Random direction*: replacing u by a random unit vector never flipped a token within the α budget (0/8 in a smoke test), so we drop it as uninformative. *Noise* (held-out only): for every edit we replace u and a with random Gaussian vectors, with a rescaled to the original $\|a\|$

Edit location	Fixed / 91	Fixed by 1 edit	Re-roll null	McNemar p
all tokens				
Decisive token	50	31	35.2	0.72
Random position	47	22	42.2	–
code tokens only				
Decisive token	49	36	31.2	0.85
Random position	47	22	40.4	–

Table 2: Per-task repair on 91 pairs (greedy fails, ≥ 1 of 10 samples passes), with up to three rank-one edits per task at layer 16. The re-roll null is the expected number of fixes if each edit were a fresh sample. McNemar p compares the two rows of each block on paired task outcomes. In the single-edit column, the p-values are 0.078 (all tokens) and 0.004 (code tokens only).

and the same α, so each rank-one term has the same norm. *Re-roll null*: a task with sample pass rate p that receives n edits would be fixed with probability $1 - (1 - p)^n$ if each edit amounted to drawing a fresh sample. Summed over tasks, this gives the fix count expected from rerolling alone.

Code-only variant. Even with the code-only prompt, about 35% of answers contain comments or docstrings. In the code-only variant we strip comments and docstrings from every passing sample, re-tokenize it, keep it only if it still passes, and then choose s among these. We also reject decisive tokens that fall inside a comment or docstring, outside the code fence, or that change only whitespace. Rejected tokens stop the episode and count as unfixed. The random-position control is restricted in the same way.

5 Results

5.1 Per-task repair

Table 2 reports per-task repair on the 91 pairs. Decisive-token edits fix 50 tasks and random-position edits fix 47. The difference is not significant (exact McNemar on paired outcomes, p=0.72). Both beat the re-roll null (35 and 42 expected fixes), but only modestly. Random-position episodes use more edits (219 flips vs. 155), so their null is higher.

Restricting the comparison to the *first* edit is more informative. A single decisive edit fixes 31 tasks, against 22 for a single random-position edit (p=0.078). In the code-only variant the gap widens to 36 vs. 22 (p=0.004; still $p < 0.02$ after Bonferroni correction over the four tests reported here). Localization is therefore not useless: the decisive token is more often the one edit that is enough. However, random-position episodes catch up by their second or third edit, so total fixes end up equal. Any trajectory-altering flip has a fair chance of reaching *some* passing completion.

5.2 Transfer to unrelated held-out tasks

If a decisive edit corrects a reusable piece of computation, then the fixing edits from some tasks should help on others. We split the 377 tasks in half at random (seeds 0, 1, 2). We sum the edits of every fixed episode whose task is in the training half, multiply by a scale λ, and decode all tasks greedily in batches of 16. We compare each held-out task's outcome to the same batched decoding without edits. Batched and unbatched greedy decoding already disagree on 14 of 377 tasks, so changes of a few tasks are noise.

Tokens	Edit	×1.0 fixed/broke	×0.25 net (seeds 0/1/2)	×0.5 net
all	Decisive	6/47	+6 / +7 / -3	–
all	Random pos.	9/42	-5 / +1 / +5	–
all	Noise	5/2	+0	–
code only	Decisive	–	-3 / +13 / +1	-10 / -13 / -21
code only	Random pos.	–	-7 / +8 / +8	-15 / -5 / -12
code only	Noise	–	+3 / -3 / -1	–

Table 3: Held-out transfer. The fixing edits of training-half tasks are summed, scaled, and applied, and the held-out half is decoded greedily. "Fixed/broke" counts held-out tasks that change from fail to pass or from pass to fail (seed 0). "Net" is fixed minus broke for seeds 0/1/2. A dash means the configuration was not run.

Table 3 gives the results. At λ=1 the summed edits are destructive. Decisive edits fix 6 held-out tasks and break 47; random-position edits fix 9 and break 42. Norm-matched noise changes almost nothing (5 fixed, 2 broken). The damage therefore comes from the *directions* u (token-output directions), not from the size of the update. At λ=0.25 net effects are small and inconsistent across seeds. Decisive edits average +3.3 (all tokens) and +3.7 (code only), against +0.3 and +3.0 for random positions. Every one of these comes with 7–25 broken tasks offset by a similar number of fixed ones. At λ=0.5 every configuration is net negative. Even on the *training* tasks, the summed edits recover only about half of the individual fixes (e.g. 12 of 21). The edits interfere with one another.

6 Transfer within a problem family

The MBPP+ result leaves two explanations open. Rank-one edits may be unable to carry anything reusable, or MBPP+ tasks may simply share too little for anything to carry over. To separate the two, we test transfer at the shortest useful distance: unseen variants of the same problem.

Templates. We wrote 32 parametric code templates. Each is a function specification with one or two values in its text, for example "returns how many non-negative integers in `lst` have exactly $\{k\}$ bits set to 1". Each template yields up to 15 distinct parameter settings, 447 variants in total. A reference implementation generates three visible asserts and a 28-assert hidden suite for every variant, and every suite was checked against its reference. The prompt format is the one used for MBPP+. The wording is fixed, and correct solutions of two variants differ only in constants.

Screening and episodes. For each variant we record the greedy answer and 10 samples at temperature 0.7. Greedy passes 226 of the 447 variants. Sixteen templates have at least three *pair* variants (greedy fails, some sample passes). The other templates are almost always or almost never solved. These 16 templates contain 123 pairs. On every pair we run the code-only episodes of Section 4, with up to three edits at layer 16, once with decisive tokens and once with random positions. Decisive episodes fix 81 of the 123 pairs, and random-position episodes fix 91.

Protocol. For each template and split seed (0, 1, 2) we draw five variants at random as the *edit set*; the rest are held out. We sum the fixing edits of the edit-set pairs, scale them by $\lambda \in \{1, 0.5\}$, and decode every held-out variant greedily and with four samples at temperature 0.7 under a

Scale	Edits	Units	Greedy pass (before→after)	Fixed / broke	Δ sample rate
×1	Decisive token	36	44/292 → 217/292	174 / 1	+0.16
	Random position	42	47/346 → 201/346	159 / 5	+0.10
	Norm-matched noise	36	44/292 → 47/292	11 / 8	-0.01
	Other template's edits	36	44/292 → 60/292	40 / 24	-0.04
×0.5	Decisive token	36	44/292 → 211/292	173 / 6	+0.20
	Random position	42	47/346 → 236/346	193 / 4	+0.19
	Norm-matched noise	36	44/292 → 45/292	8 / 7	-0.01
	Other template's edits	36	44/292 → 55/292	32 / 21	-0.01

Table 4: Transfer to unseen variants of the same template (16 templates, three splits). The fixing edits of five variants per template are summed, scaled and applied, and the template's held-out variants are decoded. "Greedy pass" and "Fixed / broke" count held-out variants. Δ sample rate is the mean change in the pass rate of four samples per variant. Random-position edits have more units because their episodes fix more pairs.

fixed sampling seed. We compare each outcome to the same decoding without edits. A *unit* is a template and split with at least one fixing edit. There are 36 units for decisive edits and 42 for random positions. Two controls are applied to the same units. *Noise* is the norm-matched noise of Section 4. *Other template* applies the decisive edits of a different template, the next one alphabetically among those with edits, from the same split. We compare configurations with a two-sided paired sign-flip permutation test over units.

Results. Table 4 shows that transfer appears. At λ=1, decisive edits raise greedy passes on held-out variants from 44 to 217 of 292, fixing 174 and breaking one. The sampled pass rate rises by 0.16. Norm-matched noise fixes 11 and breaks 8, and other templates' edits fix 40 and break 24. In paired comparisons, decisive edits beat noise in net greedy gain in 31 units and lose in none ($p < 10^{-4}$). They beat other templates' edits 25 to 1 ($p < 10^{-4}$). On the sampled pass rate, both comparisons give $p \leq 0.001$, and λ=0.5 gives the same picture. Random-position edits transfer too (159 fixed, 5 broken at λ=1; $p < 10^{-4}$ against noise). They are statistically indistinguishable from decisive edits. The paired net difference is +0.6 per unit at λ=1 (p=0.18) and −0.5 at λ=0.5 (p=0.39). For the sampled rate, the difference is +0.06 at λ=1 (p=0.03, uncorrected) and −0.02 at λ=0.5 (p=0.29). Table 5 breaks the gains down by template. Ten of the 16 templates gain 8 or more held-out variants. The other six gain at most 3, either because few of their edit-set pairs are fixed or because their edits do not carry over.

What the edits change. Of the 103 flips in fixed decisive episodes, 37 are at the first token of the function body (45 of 153 for random positions). On MBPP+, the same is true of only 10 of 65 code-only decisive flips. The most frequent flips are `return`→`result` (18 times), which replaces an immediate return expression with a named intermediate result, and `if`→`return`, which drops an opening input guard in favor of a direct expression. Both decide the overall shape of the solution. The edit's key, the MLP input a at the edited position, is shared across variants. The median cosine between the first-edit keys of two variants of the same template is 0.89, and 0.99 when both edits sit at the first body token. Across templates, it is 0.22. A rank-one edit therefore fires on held-out variants almost as strongly as on the variant it was built for, and imposes the same strategy there. Other templates present different keys, so their edits barely fire.

Template	Decisive	Random pos.	Noise	Other template
`bits_k`	+18	+17	+0	+4
`caesar`	+12	+7	+0	+0
`chunk_sums`	+21	+21	+0	+21
`clamp`	+27	+12	+0	+0
`coprime_k`	+24	+24	+4	-6
`digit_sum_base`	+1	+2	+0	+1
`every_kth`	+0	+8	+0	+0
`keep_at_most`	+8	+8	+0	+1
`kth_smallest`	+2	-1	+0	-2
`move_to_end`	+0	+0	+0	+0
`replace_kth`	+11	+11	+0	+1
`rotate_left`	+9	+9	-1	-5
`round_to`	+1	+1	+0	+1
`sum_last_digits`	+19	+20	+0	-9
`to_base`	+3	-2	+0	+3
`vowel_replace`	+17	+17	+0	+6

Table 5: Net greedy gain (fixed minus broke) on held-out variants per template at λ=1, summed over three splits.

Edits from other templates. Over all units, other templates' edits are no different from noise. The net greedy difference is +0.36 per unit (p=0.48), and the sampled rate difference is −0.03 (p=0.33). They can still hurt. Edits from `chunk_sums` lower the sampled pass rate of `caesar` from 0.20–0.28 to 0.00–0.08 in all three splits. Edits from `digit_sum_base` lower that of `coprime_k` from 0.38–0.50 to at most 0.03 and break all six of its greedy passes. The one consistent positive case is `clamp`→`chunk_sums`, with a net gain of +6 to +8 in each of the three splits. Both templates operate on a list, and both make their failing decision at the first body token. Clamp's edits suppress an immediate `return` there, and chunk_sums' own fixes mostly replace an opening `if`. So transfer across templates seems to require a shared decision point at a similar context, and pushing the model off its failing first choice can be enough.

7 Discussion

Context-keyed strategy switches. The two transfer results fit one picture. A rank-one edit adds a fixed output direction whenever the layer input resembles its key. When the decisive decision falls at a context that recurs, such as the first body token of the same specification with different constants, the edit fires there and imposes the same choice, so it transfers. MBPP+ tasks rarely share such contexts. Their summed edits fire weakly and inconsistently on unrelated prompts and interfere with one another, which gives the symmetric fix/break churn of Table 3. Seen this way, the edits do not correct a reusable mechanism. They are lookup-table entries keyed on context, and they transfer as far as their key does. This also explains why the decisive token barely matters. Any flip that dislodges the failing opening move at a shared context works, and random disagreement positions often land on the same early structural tokens. This mirrors the finding of Hase et al. [2023] for factual knowledge, where the location found by causal tracing does not predict where editing works best.

Implications for self-editing. The pilot shows that a small model cannot choose useful edits from logit-lens labels. The main experiments show what a model *can* do without choosing anything. Given its own passing samples and a verifier, it can install edits that improve its greedy behavior on a whole family of related problems, with almost no damage inside the family. This is a narrow but genuine form of self-improvement, comparable to caching a strategy. It does not need precise localization. It does not extend to unrelated tasks, and edits keyed on one family can harm another. A practical system would need routing or gating so that edits fire only where their key belongs, and it should be compared with cheaper alternatives such as putting a solved example in the prompt or fine-tuning on the model's own passing samples [Zweiger et al., 2025]. Constrained editors such as AlphaEdit [Fang et al., 2025] could reduce cross-family interference.

8 Limitations

We study one small model (0.8B), one layer, and the simplest rank-one editor. Larger models, covariance-aware editors such as ROME and MEMIT, and multi-layer edits could all behave differently. The templates are synthetic and were chosen after screening for mixed pass rates. Several have only 5–8 variants, so some units hold out as few as three variants. We have not yet compared weight edits with in-context alternatives, such as showing a solved variant in the prompt, or with fine-tuning on the same passing samples. Without these comparisons, we cannot claim that weight edits are the best way to use this signal. Episodes use the hidden tests to choose passing samples and to decide when to stop, so the method assumes a reliable verifier. On MBPP+, the 91 pairs and three held-out splits give limited power, and batched decoding adds noise of about ± 7 tasks. Our edits require a passing sample, so they cannot help on tasks the model never solves (156 of 247 MBPP+ failures). We report every configuration we ran, including the ones that failed.

9 Conclusion

A binary search over forced prefixes finds the token decision that separates a failing answer from a passing one, and a rank-one edit can flip it. On MBPP+, this repairs about half of the solvable failures of a 0.8B code model. Edits at random disagreement positions repair almost as many, and neither kind transfers to unrelated tasks. On unseen variants of the same problem, the same edits do transfer, raising greedy passes from 44 to 217 of 292, and neither noise nor other problems' edits come close. The edits behave as strategy switches keyed on context, so they generalize exactly as far as that context recurs. Self-editing of this kind can improve a model on a problem family it has partly solved. Locating the decisive token is not what makes it work, and it does not by itself yield general self-improvement.

Reproducibility and AI assistance

All code, run logs, and edit deltas are available at
`https://github.com/avijitbhuin21/decisive-token-self-edits`.
The MBPP+ numbers are regenerated from the logs by `scripts/paper_stats.py`, and the template numbers by `scripts/variant_stats.py`. The templates are defined in `selfedit/variants.py`. The experiments were run on a single Apple M5 laptop. An AI coding assistant helped write the experiment code and draft this paper. The author reviewed all analyses and takes responsibility for the content.

References

Marah Abdin et al. Phi-4 technical report, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

Junfeng Fang et al. AlphaEdit: Null-space constrained knowledge editing for language models. In *International Conference on Learning Representations*, 2025. arXiv:2410.02355.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of EMNLP*, 2021.

Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2301.04213.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.01210.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2202.05262.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations*, 2023. arXiv:2210.07229.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale. In *International Conference on Learning Representations*, 2022. arXiv:2110.11309.

nostalgebraist. interpreting GPT: the logit lens. LessWrong blog post, 2020.

Qwen Team. Qwen3.5-0.8B. Model release; MLX conversion `mlx-community/Qwen3.5-0.8B-bf16`, 2026.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models, 2025.